Генераторы случайных чисел

Лекция 5

Содержание

1 Определение, классификация, приложения

2 Примеры генераторов случайных чисел

3 Параллельные вычисления

4 Примеры генерации распределений

5 Примеры программных решений

https://t.me/+2kGsBQZcdBQxNDAy

Определение, классификация, приложения

> “Anyone who attempts to generate random numbers by deterministic means is, of course, living in a state of sin.”
> John von Neumann

Определения

- Генератор случайных чисел (RNG) – объект для получения последовательности случайных чисел
  - Числа невозможно предсказать точнее, чем в случае произвольного выбора [ссылка]
- Генератор случайных битов (RBG) – устройство или алгоритм для получения последовательности битов, которые «похожи» на несмещенные и статистически независимые [NIST SP 800-90A]
  - **Детерминированный (DRBG)**
    - Алгоритм, который производит одни и те же выходы для заданных входов
    - Может иметь доступ к источнику энтропии (например, к термическому шуму) для получения seed
    - Обладает набором функций для (де-)инстацирования, генерации битов, ре-инициализации
  - **Недетерминированный (true random number generators, TRNG)**
    - Имеет постоянный доступ к источнику энтропии
    - Производит последовательности битов, являющиеся практическим приближением идеальных: биты не являются предсказуемыми, смещенными, зависимыми от других битов; значение каждого бита является равновероятным и не зависит от знания значений других битов; последовательность из n битов содержит n битов энтропии (Энтропия сл. величины X: $H(X) = -\sum_x P(x) log P(x)$, где суммирование идет по всевозможным значениям X, $P(x)$ - вероятность выпадение значения x)

Определение, классификация, приложения

> “Anyone who attempts to generate random numbers by deterministic means is, of course, living in a state of sin.”
> John von Neumann

Детерминированные генераторы

- **Псевдослучайные (PRNG)** производит последовательности, которые только аппроксимируют свойства случайных чисел (значение, имеющее одинаковые шансы быть выбранным из области значений), DRBG
- **Квазислучайные (QRNG)** производит последовательности, такие что для некоторого множества B кол-во попавших в B чисел пропорциональное его объему:
  - $(x_i)_{i=1,\dots,n}$ – последовательность чисел из $\mathrm{I} = [0,1)$, $[\mathrm{a}, \mathrm{b}) \in I$
  - $\sup\limits_{[\mathrm{a},\mathrm{b})}\{\frac{\{x_1,\dots,x_n\}\cap[a,b)}{n} - (b-a)\} \leq K$, где K – некоторый порог

Криптографические генераторы

- Обладают свойствами, позволяющими их использовать в криптографии:
  - Генерация ключей, протоколы аутентификации, вычисление хэшей, токинезация
- 1) TRNG, 2) PRNG, инициализированный с помощью TRNG
- С точки зрения вычислительной сложности:
  - Не найдется теста, который за полиномиальное время мог бы предсказать любой бит в последовательности с шансами успеха > 0.5

Определение, классификация, приложения

“Anyone who attempts to generate random numbers by deterministic means is, of course, living in a state of sin.”
John von Neumann

Примеры приложений

- Симуляции (физика, финансы, …)
- Лотереи, игры
- Безопасность (автономные автомобили, телеком, космос,…)
- Машинное обучение
- …

Д. Кнут, Искусство программирования, т2

Примеры генераторов случайных чисел

Детерминированный RNG – структура $(\boldsymbol{S}, \boldsymbol{\mu}, \boldsymbol{f}, \boldsymbol{U}, \boldsymbol{g})$

- $\boldsymbol{S}$ – конечное пространство состояний генератора
- $\boldsymbol{\mu}$ – вероятностное распределение на S для выбора начального состояния $\boldsymbol{s_0}$ (seed)
- $\boldsymbol{f}: S \to S$ – **детерминированная** функция перехода
- $\boldsymbol{U}$ – конечное пространство выходов генератора, например $U = [0,1)$
- $\boldsymbol{g}: S \to U$ – **детерминированная** функция генерации выхода u

Некоторые свойства

- **Эволюция генератора**: некоторое рекуррентное соотношение $s_{i+1} = f(s_i)$
- **Генерация выхода:** $u_i = g(s_i)$, последовательность $\{u_i\}$ –сл. числа, произведенные генератором
- **Период генератора:** наименьшее положительное j, для которого $s_{j+l} = s_l$
  - $s_{j+l} = s_l$ из-за конечности S, далее последовательности s_* и u_* повторяются
  - $j \leq |S|, j \leq 2^k$, где k –количество битов, описывающих состояние s_*

Pierre L'Ecuyer, Random Number Generation
https://www.iro.umontreal.ca/~lecuyer/myftp/papers/handstat.pdf

Примеры генераторов случайных чисел

Некоторые критерии качества RNG

- **Большой период:** доказывается математически
- **Эффективность:** 1) объем используемой памяти для состояния, 2) скорость вычислений f и g
- **Повторяемость:** воспроизводимость той же самой последовательности сколь угодно раз
- **Поддержка параллельных вычислений:** исходная последовательность декомпозируется на непересекающиеся подпоследовательности для их использования на параллельных системах

$\{u_i\}_{i\geq 0}$ **последовательность независимых равномерно распределенных на** $(0,1)$ **величин**$\Leftrightarrow$ для любых i и t вектор $u = (u_i, \dots, u_{i+t-1})$ равномерно распределен в гиперкубе $H = (0,1)^t$

- $\Psi(I) = \{(u_{i_1}, \dots, u_{i_t}) | s_0 \in S\}$, где $I = \{i_1, \dots, u_{i_t}\}$ - аппроксимация H
- Большой период генератора необходим для построения достаточно хорошей аппроксимации H
- **Мера аппроксимации:** несоответствие (discrepancy)
  - $u = (u_0, \dots, u_{N-1}) \in H$, $Hs \in H$, $A(Hs, u)$ - количество точек u, попавших в Hs
  - $D_N(u) = \sup\limits_{Hs} \left| \frac{A(Hs,u)}{N} - \lambda(Hs) \right|$, $\lambda(Hs)$ - мера Лебега множества Hs

H Niederreiter, Low-Discrepancy and Low-Dispersion Sequences, Journal of Number Theory, 30, 51-70 (1988)

Примеры генераторов случайных чисел

Линейные рекурсии по модулю m

Мультипликативный рекурсивный генератор

- $x_i = a_1 x_{i-1} + \cdots + a_k x_{i-k} \; mod \; m$, где k – порядок генератора, m - модуль генератора, $u_i = \frac{x_i}{m}$
- Для простого числа m возможно подобрать коэффициенты $\{a_i\}_{i=1,\ldots,k}$ так, чтобы период генератора $T = m^k - 1$
  - Характеристический полином $P(z) = z^k - a_1 z^{k-1} - \cdots - a_k$ - примитивный над кольцом вычетов по модулю m Z_m ($Z_m = \{0,..,m-1\}$ с операциями по модулю m)

Линейный конгруэнтный генератор

- $x_i = a x_{i-1} mod \; m$

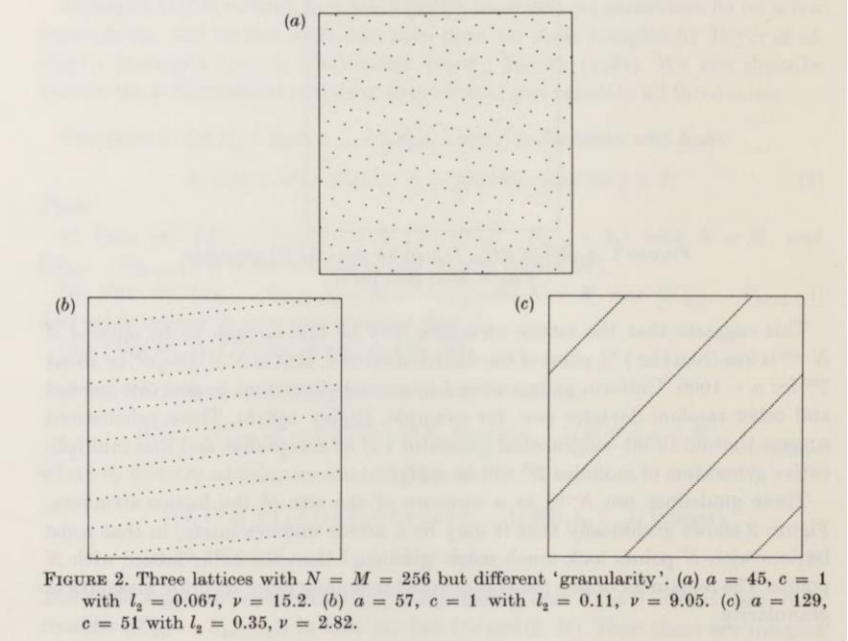


FIGURE 2. Three lattices with $N = M = 256$ but different 'granularity'. (a) $a = 45$, $c = 1$ with $l_2 = 0.067$, $\nu = 15.2$. (b) $a = 57$, $c = 1$ with $l_2 = 0.11$, $\nu = 9.05$. (c) $a = 129$, $c = 51$ with $l_2 = 0.35$, $\nu = 2.82$.

https://royalsocietypublishing.org/doi/epdf/10.1098/rspa.1983.0103

Решетка генератора:
- Параллельные гиперплоскости, располагающиеся на одинаковом расстоянии друг от друга
- Гиперплоскости содержат выход генератора

Расстояние между гиперплоскостями – мера качества генератора

Примеры генераторов случайных чисел

Линейные рекурсии по модулю m

Комбинированные рекурсивные генераторы

- $x_{j,i} = a_{j,1}x_{i-1} + \cdots + a_{j,k}x_{i-k} \; mod \; m_j, j = 1, \dots, J$
- Примеры g:
  - $u_i = \frac{z_i}{m_1}, z_i = b_1 x_{1,i} + \cdots + b_J x_{J,i} \; mod \; m_1$
  - $w_i = b_1 \frac{x_{1,i}}{m_1} + \cdots + b_J \frac{x_{J,i}}{m_J} \; mod \; 1$
  - Расстояние между u_i и w_i не превосходит некоторого порога при определенных условиях

Практический способ реализации генераторов этого класса

- Выбрать несколько ненулевых коэффициентов $a_{j,i}$ для каждого j и достаточно большие m_j
- Если m_j - простые, и каждая компонента имеет максимальный период $T_j = m_j^k - 1$, тогда период комбинации не превышает $\frac{T_1 * \cdots * T_J}{2^{J-1}}$
- Параметры для комбинации и С код: Pierre L'Ecuyer. Fast combined multiple recursive generators with multipliers of the form a = ±2q ± 2r

Примеры генераторов случайных чисел

Линейные рекурсии по модулю 2

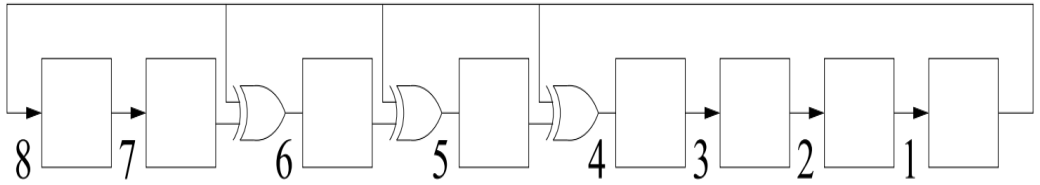
- F_2 – поле Галуа 2-го порядка, состоящее из двух элементов, 0 и 1
- $x_i = A\,x_{i-1}$, $x_i = \left(x_{i,0}, \dots, x_{i,k-1}\right)^T \in F_2^k$ - вектор состояния, $A = A_{kxk}$ - матрица перехода с элементами в F_2
- $y_i = B\,x_i$, $y_i = \left(y_{i,0}, \dots, y_{i,w-1}\right)^T \in F_2^w$ - выходной вектор, $\mathrm{B} = B_{wxk}$ - матрица перехода с элементами в F_2
- $u_i = \sum_{l=1}^{w} y_{i,l-1}\,2^{-l}$, $u_i \in [0,1)$
- Решетка генератора определяется в пространстве формальных степенных рядов

Pierre L'Ecuyer and Francois Panneton
Fast Random Number Generators based on linear recurrences modulo 2: overview and comparison

Примеры генераторов случайных чисел

Линейные рекурсии по модулю 2

- **Tausworthe Linear Feedback Shift Register (LFSR)**
- $A_0 = \begin{pmatrix} & 1 & & \\ & & \dots & \\ & & & 1 \\ a_1 & a_2 & \dots & a_k \end{pmatrix}, A = A_0^s$
- Матрица B содержит первые w строк единичной матрицы размером kxk
- s – параметр генератора
- $x_i = a_1 x_{i-1} + \dots + a_k x_{i-k} \; mod \; 2$
- $u_i = \sum_{l=1}^{w} x_{is+l-1} 2^{-l}$



https://www.physics.otago.ac.nz/reports/electronics/ETR2012-1.pdf

Robert C. Tausworthe, Random Numbers Generated by Linear Recurrence Modulo Two

Pierre L'Ecuyer and Francois Panneton
Fast Random Number Generators based on linear recurrences modulo 2: overview and comparison

Примеры генераторов случайных чисел

Линейные рекурсии по модулю 2

- **Twisted Generalized Feedback Shift Register (TGFSR)**
  - $A = A_{pqxpq} = \begin{pmatrix} & & I_p & S \\ I_p & & & \\ & I_p & & \\ & & \dots & \\ & & & I_p \end{pmatrix}$, где I_p - единичная матрица размера p , $S = S_{pxp} = A_0^T$ (A_0 определена на предыдущем слайде)
  - $w = p$
  - $T = 2^{pq} - 1$
  - Матрица B выбирается так, чтобы улучшить равномерность получаемых последовательностей (реализует tempering операции)

Комбинированные LFSR

- Линейная комбинация y_i

Pierre L'Ecuyer and Francois Panneton
Fast Random Number Generators based on linear recurrences modulo 2: overview and comparison

Примеры генераторов случайных чисел

Линейные рекурсии по модулю 2

- **Mersenne Twister (вариант TGFSR)**
- $x_{k+n} = x_{k+m} \oplus (x_k^u \mid x_{k+1}^l)A, k = 0,1,2, \dots$
- x^u - старшие $w - r$ биты числа x, x^l - младшие r битов числа x
- $A = \begin{pmatrix} & 1 & & & \\ & & 1 & & \\ & & & \dots & \\ & & & & 1 \\ a_{w-1} & a_{w-2} & \dots & \dots & a_0 \end{pmatrix}$
- $xA = \begin{cases} shiftright(x), & \text{если } x_0 = 0 \\ shiftright(x) \oplus a, & \text{если } x_0 = 1 \end{cases}$
- Tempering xB
  - $y = x \oplus (x \gg u)$, $y = y \oplus ((y \ll s) \,\&\, b)$
  - $y = y \oplus ((y \ll t) \,\&\, c)$, $z = y \oplus (y \gg l)$
- $T = 2^{19937} - 1$, 623-распределенная последовательность с точностью до 32 битов

$trunk(x, v)$ – ведущие v битов целого числа x
$(trunk(x_i, v), \dots, trunk(x_{i+k-1}, v), i < T$ – kv–битный вектор

Последовательность x_i псевдослучайных чисел периода T, k-распределена с точностью до v битов, если каждая из 2^{kv} возможных комбинаций битов случается одинаковое кол-во раз за период T (за исключением нулевой последовательности)

Для каждого v k(v) – максимальное значение, при котором последовательность k(v) распределена с точностью до v

w=32, n =624, m=397, r=31
A = 9908B0DF
u=11, s=7, b = 9D2C5680
t=15, c = EFC60000
l=18
p=nw-r – экспонента Mersenne

M Matsumoto, T Nishimura
Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator
https://dl.acm.org/doi/pdf/10.1145/272991.272995
https://github.com/notr1ch/opentdm/blob/master/mt19937.c

Примеры генераторов случайных чисел

Philox (часть C++26)

- $s_n = b(n)$, где b – некоторая функция, n – p –битный счетчик (например, $p = 128$)
- Генератор псевдослучайных чисел на базе счетчика с периодом $T = 2^p$, если b удовлетворяет некоторым требованиям. Функция $b = b_k$ может быть определена на базе блочных шифров, где k- элемент пространства ключей K, b_k - элемент семейства биекций
- Обобщение ранее данного определения генератора:
  - $\boldsymbol{f}: S \to S$ – **детерминированная** функция перехода
  - $\boldsymbol{g}: K\ x\ Z_j\ x\ S \to U$ – **детерминированная** функция генерации выхода u, $j \in Z_J$ - индекс для получения чисел из внутреннего состояния (например, $J = 4$)
- Состояние малого размера, эффективная поддержка параллельных вычислений

Алгоритм Philox4x32:

- c_n - 128-битный счетчик
- $c_n^i, i = 0, \dots, 3$ - 32-битные части c_n (c_n^3 - старшая часть)
- K^0 и K^1 - 32-битные ключи
- $mulhi(a, b) = \frac{a\ x\ b}{2^W}$, $mullo(a, b) = a\ x\ b\ mod\ 2^W$, W-размер слова состояния вычислительного устройства
- $m := 0, c_0 := 0$
- $k^0 := K^0, k^1 := K^1$
- $for\ i := 1, \dots, 10$
  - $X_0 := c_n^0$
  - $X_2 := c_n^2$
  - $c_n^0 := mulhi(X_2, 0xCD9E8D57) \oplus k^0 \oplus c_n^1$
  - $c_n^1 := mullo(X_2, 0xCD9E8D57)$
  - $c_n^2 := mulhi(X_0, 0xD2511F53) \oplus k^1 \oplus c_n^3$
  - $c_n^3 := mullo(X_0, D2511F53)$
  - $k^0 := k^0 + 0x9E3779B9$
  - $k^1 := k^1 + 0xBB67AE85$
- $endfor$
- $\{u_m, u_{m+1}, u_{m+2}, u_{m+3}\} := \{c_n^0, c_n^1, c_n^2, c_n^3\}, m := m + 4$
- $c_n := c_{n-1} + 1$
- Обновление частей c_n - функция (сеть) Фейстеля

John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel Random Numbers: As Easy as 1,2,3
https://www.thesalmons.org/john/random123/papers/random123sc11.pdf

Параллельные вычисления

Поддержка параллельных вычислений

- **Skip-ahead**
  - Последовательность $(x_i)_{i\geq 0}$ разбивается на непересекающиеся блоки чисел фиксированного размера
  - Блоки используются для вычислений независимо на исполняемых устройствах

$x_0, \dots, x_{k-1}$ | $x_k, \dots, x_{2k-1}$ | $x_{2k}, \dots x_{3k-1}$

- **Leapfrog**
  - Последовательность разбивается на непересекающиеся подпоследовательности
  - Подпоследовательности используются для вычислений независимо на исполняемых устройствах

x_0 | x_1 | x_2 | x_3 | x_4 | x_5 | x_6 | x_7 | x_8

- **Семейство генераторов**
  - Генераторы используют один и тот же алгоритм, его параметры обеспечивают независимость и равномерную распределенность последовательностей
  - Генераторы используются независимо на вычислительных устройствах

$x_i = a_1 x_{i-1} \bmod m_1$ | $x_i = a_2 x_{i-1} \bmod m_2$ | $x_i = a_2 x_{i-1} \bmod m_2$

Примеры генерации распределений

Обратная функция

- $F = F(x)$ –функция распределения (экспоненциального, нормального, ...)
- u – случайная величина, имеющая равномерное распределение на интервале $[0,1)$
- $F^{-1}(y) = \inf_x\{\mathrm{F(x)} \geq \mathrm{y}\}$
- $\xi = F^{-1}(u)$ – случайная величина, имеющая распределение F

Примеры

- **Экспоненциальное распределение**: $F(x;\lambda) = 1 - e^{-\lambda x}, x \geq 0, \lambda > 0$
  - $F^{-1}(y;\lambda) = -\frac{1}{\lambda}\ln(1-u)$
- **Нормальное распределение:** $F(x;a,\sigma) = \frac{1}{\sigma\sqrt{2\pi}}\int_{-\infty}^{x} e^{-\frac{(y-a)^2}{2\sigma^2}}dy$, $F(x;0,1)$ - стандартное нормальное распределение
  - $F(x;a,\sigma) = \frac{1}{2}[1 + \mathrm{erf}(\frac{x-a}{\sigma\sqrt{2}})]$, $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}dy$ - функция ошибки
  - $F^{-1}(y;a,\sigma) = \sigma\sqrt{2}\,\mathrm{erf}^{-1}(2u-1) + a$, $\mathrm{erf}^{-1}(x) = \frac{\sqrt{\pi}}{2}\left(x + \frac{\pi}{12}x^3 + \frac{7\pi^2}{480}x^5 + \frac{4369\,\pi^4}{5806080}x^9 + \cdots\right)$

Примеры генерации распределений

Свойства распределений

- Случайные величины g и l связаны функциональной зависимостью $l = L(g)$
- Если известен эффективный генератор для g, то его возможно использовать для генерации l

Пример

- **Логнормальное распределение:** $F(x; a, \sigma) = \int_0^x \frac{1}{y\sigma\sqrt{2\pi}} e^{-\frac{(\ln y - a)^2}{2\sigma^2}} dy,\ x \geq 0$
- l – сл. величина, имеющая логнормальное распределение
- $l = exp(g)$, где g имеет нормальное распределение с параметрами a, σ

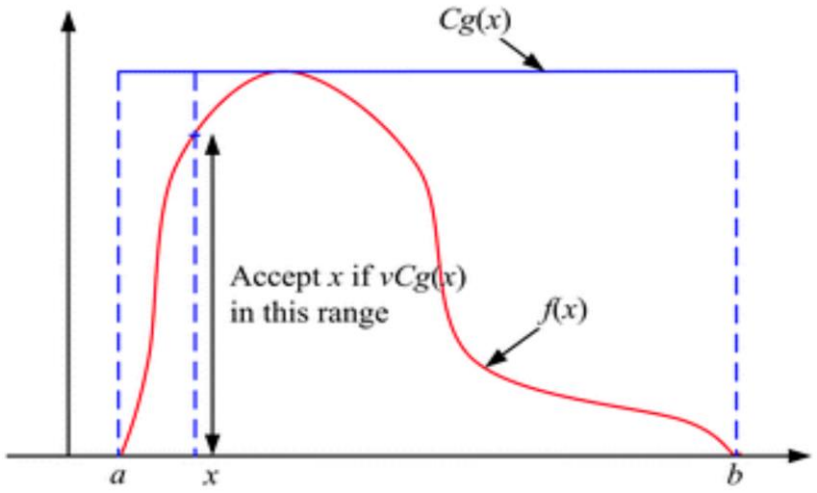
Трансформации

- (u_1, u_2) – пара независимых равномерно распределенных на [0,1) величин
- $z_1 = \sqrt{-2\ln(u1)}\cos(2\pi u_2), z_2 = \sqrt{-2\ln(u1)}\sin(2\pi u_2)$
- (z_1, z_2) – пара независимых стандартных Гауссовых величин

Примеры генерации распределений

Принятие-отклонение (acceptance-rejection)

- Строим генератор распределения $F(x) = \int_{-\infty}^{x} f(y)dy$
- Известен генератор распределения $\mathrm{G}(x) = \int_{-\infty}^{x} g(y)dy$
- $g(x)$ аппроксимирует $\mathrm{f}(x)$: $\sup\limits_{x} \frac{f(x)}{g(x)} \leq c$
- **Общая схема**
  - Генерируем y из G
  - Генерируем u из равномерного на $[0,1)$ распределения, u не зависит от y
  - Если $u \leq \frac{\mathrm{f(y)}}{cg(y)}$, то возвращаем (принимаем) y, иначе новая итерация (отклоняем)
  - Ожидаемое кол-во итераций алгоритма до получения $x - \mathrm{c}$



https://www.researchgate.net/figure/Illustration-of-the-acceptance-rejection-method_fig18_295100042

Генератор Гамма распределения с параметром a
f(y) = y^(a-1) exp(-y) / Gamma(a), y> 0
Эффективность близка к 1

```
(1) Setup: d=a-1/3, c=1/sqrt(9d).

(2) Generate: v=(1+c*x)^3, with x standard normal.

(3) if v>0 and log(UNI) < 0.5*x^2+d-d*v+d*log(v) return d*v

(4) go back to step 2.
```

https://dl.acm.org/doi/pdf/10.1145/358407.358414

Примеры генерации распределений

Дискретные схемы

- Генератор Бернулли: $x = 1$ с вероятностью $p \in [0,1]$, 0 – в противном случае
  - Генерируем u из равномерного на $[0,1)$ распределения
  - Если $u < \mathrm{p}$, возвращаем 1, иначе 0

Генерация двумерного распределения с заданным коэффициентом корреляции

- (X_1, X_2) – пара независимых стандартных нормальных величин
- Построить (Y_1, Y_2) пару стандартных нормальных величин с заданным коэффициентом корреляции ρ
  - $Y_1 = X_1$
  - $Y_2 = \rho X_1 + \sqrt{1-\rho^2}\, X_2$

Примеры программных решений

Генераторы

- C++, BOOST: LCG(16807, 2^31-1), LCG(48271, 2^31-1), T = 2^31-1, PHILOX
- oneAPI MKL, cuRAND, AOCL RNG: MRG32K3A : 2 компоненты, в каждой компоненте 3 слагаемых в рекурентном уравнении, $T = 2^{191}$
- MKL, cuRAND, AOCL RNG, BOOST, IMSL: MT19937/MT2203/MTGP/PHILOX
- BOOST: Taus88 (комбинированный LFSR)
- NAG, MKL: Wichmann-Hill – семейство 273 комбинированных генераторов

Распределения

- Пакеты непрерывных и дискретных распределений содержатся в коммерческих решениях и решениях с открытым кодом

Домашнее задание

На выбор один пункт

- Реализовать бенчмарк «Монте Карло вычисление числа pi» с использованием любого генератора непрерывного распределения и кол-вом векторов 100 миллионов
  - Опционально: распараллелить вычисления
- Реализовать генератор Бернулли с параметром $p = 2^{-40}$
- Реализовать генератор X^2 распределения с n степенями свободы
- Реализовать генератор биномиального распределения с параметрами (p, n)
- Построить n-мерный стандартный нормальный вектор с заданной матрицей корреляций

Дополнительно на выбор один пункт

- Показать зависимость нормального и логнормального распределений (с16)
- Показать справедливость схемы генерации двумерного распределения с заданным коэффициентом корреляции